

Pandas

- Llibreria d'anàlisi de dades en Python.
- Ofereix DataFrames (estructures de dades tabulars).
- Potent per a la manipulació i exploració de dades.
- Normalment s'importa com a `pd`.

```
In [1]: import pandas as pd
```

DataFrames

- Els DataFrames es representen com a estructures bidimensionals.
- Tenen índexs, noms de columna i un valor per a cada parella índex-columna.
- Documentació: <https://pandas.pydata.org/docs/reference/frame.html>

```
In [2]: data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva", "Frank", "Grace"],
    "Age": [22, 30, 35, 28, 35, 50, 22],
    "City": ["NY", "LA", "SF", "NY", "NY", "SF", "LA"],
    "Salary": [100, 200, 300, 250, 350, 500, 80],
    "Position": ["Engineer", "Manager", "CEO", "Engineer", "Manager", "Engineer", "Intern"],
    "Years of Experience": [2, 5, 10, 4, 12, 15, 1]
}

# Crea un DataFrame a partir de les dades d'un diccionari
df = pd.DataFrame(data)
df
```

```
Out[2]:
```

	Name	Age	City	Salary	Position	Years of Experience
0	Alice	22	NY	100	Engineer	2
1	Bob	30	LA	200	Manager	5
2	Charlie	35	SF	300	CEO	10
3	David	28	NY	250	Engineer	4
4	Eva	35	NY	350	Manager	12
5	Frank	50	SF	500	Engineer	15
6	Grace	22	LA	80	Intern	1

```
In [3]: type(df)
```

```
Out[3]: pandas.core.frame.DataFrame
```

```
In [4]: # Mida del dataframe (files, columnes)  
df.shape
```

```
Out[4]: (7, 6)
```

```
In [5]: # Nombre de files  
len(df)
```

```
Out[5]: 7
```

```
In [6]: # Mida total del dataframe  
df.size
```

```
Out[6]: 42
```

```
In [7]: # Noms de les columnes  
df.columns
```

```
Out[7]: Index(['Name', 'Age', 'City', 'Salary', 'Position', 'Years of Experi  
ence'], dtype='object')
```

```
In [8]: # Noms de les columnes  
df.columns.to_list()
```

```
Out[8]: ['Name', 'Age', 'City', 'Salary', 'Position', 'Years of Experience']
```

```
In [9]: # Tipus de les columnes  
df.dtypes
```

```
Out[9]:      Name      object  
      Age      int64  
      City      object  
      Salary    int64  
      Position  object  
      Years of Experience  int64  
      dtype: object
```

```
In [10]: # Índexs per defecte  
df.index
```

```
Out[10]: RangeIndex(start=0, stop=7, step=1)
```

Primer i últim

Accedeix a les primeres files o a les últimes.

```
In [11]: # Per defecte accedeix a les primeres 5 files  
df.head()
```

```
Out[11]:
```

	Name	Age	City	Salary	Position	Years of Experience
0	Alice	22	NY	100	Engineer	2
1	Bob	30	LA	200	Manager	5
2	Charlie	35	SF	300	CEO	10
3	David	28	NY	250	Engineer	4
4	Eva	35	NY	350	Manager	12

```
In [12]: # Accedeix a les primeres 3 files  
df.head(3)
```

```
Out[12]:
```

	Name	Age	City	Salary	Position	Years of Experience
0	Alice	22	NY	100	Engineer	2
1	Bob	30	LA	200	Manager	5
2	Charlie	35	SF	300	CEO	10


```
In [13]: # Accedeix a les últimes 2 files  
df.tail(2)
```

```
Out[13]:
```

	Name	Age	City	Salary	Position	Years of Experience
5	Frank	50	SF	500	Engineer	15
6	Grace	22	LA	80	Intern	1

Accés a columnes

```
In [14]: # Accedir a la columna 'Name'  
df['Name']
```

```
Out[14]: 0      Alice  
1       Bob  
2    Charlie  
3     David  
4       Eva  
5     Frank  
6     Grace  
Name: Name, dtype: object
```

```
In [15]: # Com a tipus `list`  
df['Name'].to_list()
```

```
Out[15]: ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace']
```

```
In [16]: # Accedir a múltiples columnes  
df[['Name', 'Salary']]
```

```
Out[16]:
```

	Name	Salary
0	Alice	100
1	Bob	200
2	Charlie	300
3	David	250
4	Eva	350
5	Frank	500
6	Grace	80

Series

- Array unidimensional etiquetat.
- Pot contenir qualsevol tipus de dades.

```
In [17]: # Accedir a una columna retorna un objecte Series (com un array 1D amb índexs)  
df['City']
```

```
Out[17]: 0    NY  
         1    LA  
         2    SF  
         3    NY  
         4    NY  
         5    SF  
         6    LA  
         Name: City, dtype: object
```

```
In [18]: type(df['City'])
```

```
Out[18]: pandas.core.series.Series
```

```
In [19]: # Accedir al primer element de la columna  
df['City'][0]
```

```
Out[19]: 'NY'
```

Modificar columnes

Modifica els valors de les columnes amb qualsevol operació vàlida.

```
In [20]: # Obtenir un Series amb + 5  
df['Age'] + 5
```

```
Out[20]:  
0    27  
1    35  
2    40  
3    33  
4    40  
5    55  
6    27  
Name: Age, dtype: int64
```

```
In [21]: # EL DF no es modifica!  
df['Age'].head()
```

```
Out[21]:  
0    22  
1    30  
2    35  
3    28  
4    35  
Name: Age, dtype: int64
```

```
In [22]: # Sobreescrivere-ho per a canvis persistents  
df['Age'] = df['Age'] + 5
```

```
In [23]: df['Age'].head()
```

```
Out[23]:
```

0	27
1	35
2	40
3	33
4	40

Name: Age, dtype: int64

In [24]: *# Exemple amb strings*

```
df['Name'] + ' - ABC'
```

Out[24]:

0	Alice - ABC
1	Bob - ABC
2	Charlie - ABC
3	David - ABC
4	Eva - ABC
5	Frank - ABC
6	Grace - ABC

Name: Name, dtype: object

Noves columnes

```
In [26]: # Crear una nova columna
df['Phone'] = [''.join(map(str, np.random.randint(0, 10, size=9)))
               for _ in range(len(df))]
df
```

```
Out[26]:
```

	Name	Age	City	Salary	Position	Years of Experience	Phone
0	Alice	27	NY	100	Engineer	2	787167724
1	Bob	35	LA	200	Manager	5	110996078
2	Charlie	40	SF	300	CEO	10	902939169
3	David	33	NY	250	Engineer	4	282966260
4	Eva	40	NY	350	Manager	12	803645337
5	Frank	55	SF	500	Engineer	15	077507963
6	Grace	27	LA	80	Intern	1	152001734

Índexs

- Els índexs són identificadors **únics** per a cada fila.
- Per defecte, el rang és [0, longitud del dataframe - 1].
- Mentre els seus valors siguin únics, podem fer servir qualsevol valor com a índex.

In [27]:

```
df
```

Out[27]:

	Name	Age	City	Salary	Position	Years of Experience	Phone
0	Alice	27	NY	100	Engineer	2	787167724
1	Bob	35	LA	200	Manager	5	110996078
2	Charlie	40	SF	300	CEO	10	902939169
3	David	33	NY	250	Engineer	4	282966260
4	Eva	40	NY	350	Manager	12	803645337
5	Frank	55	SF	500	Engineer	15	077507963
6	Grace	27	LA	80	Intern	1	152001734

```
In [28]: # Establrir 'Name' com a index
df.set_index('Name', inplace=True)
df
```

Out[28]:

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Bob	35	LA	200	Manager	5	110996078
Charlie	40	SF	300	CEO	10	902939169
David	33	NY	250	Engineer	4	282966260
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963
Grace	27	LA	80	Intern	1	152001734

```
In [29]: # Restablir a l'índex per defecte
df.reset_index(inplace=True)
df
```

```
Out[29]:
```

	Name	Age	City	Salary	Position	Years of Experience	Phone
0	Alice	27	NY	100	Engineer	2	787167724
1	Bob	35	LA	200	Manager	5	110996078
2	Charlie	40	SF	300	CEO	10	902939169
3	David	33	NY	250	Engineer	4	282966260
4	Eva	40	NY	350	Manager	12	803645337
5	Frank	55	SF	500	Engineer	15	077507963
6	Grace	27	LA	80	Intern	1	152001734

Accés a files

loc

Indexació basada en etiquetes.

```
In [30]: # Establir 'Name' com a índex  
df.set_index('Name', inplace=True)
```

```
In [31]: # Accedir als atributs de l'índex 'Alice'  
df.loc['Alice']
```

```
Out[31]: Age                27  
City                NY  
Salary                100  
Position            Engineer  
Years of Experience      2  
Phone              787167724  
Name: Alice, dtype: object
```

```
In [32]: # Accedir a un atribut concret de l'índex 'Alice'  
df.loc['Alice', 'City']
```

```
Out[32]: 'NY'
```

iloc

Indexació basada en la posició.

```
In [33]: # Accedir als atributs de la segona fila (base 0), que és la d'en Bob  
df.iloc[1]
```

```
Out[33]:
```

Age	35
City	LA
Salary	200
Position	Manager
Years of Experience	5
Phone	110996078

Name: Bob, dtype: object

```
In [34]: # Accedir a un atribut concret (també per índex base 0)  
df.iloc[1, 0]
```

```
Out[34]: np.int64(35)
```

```
In [35]: # Accedir a files amb un rang d'índexs
df.iloc[0:2]
```

```
Out[35]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Bob	35	LA	200	Manager	5	110996078

```
In [36]: # Accedir a files i columnes amb rangs d'índexs
df.iloc[0:2, 1:3]
```

```
Out[36]:
```

	City	Salary
Name		
Alice	NY	100
Bob	LA	200

Filtratge

- Podem filtrar dades amb `Series` booleans.
- Els valors `True` es conserven, els `False` es descarten.

In [37]:

```
df
```

Out[37]:

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Bob	35	LA	200	Manager	5	110996078
Charlie	40	SF	300	CEO	10	902939169
David	33	NY	250	Engineer	4	282966260
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963
Grace	27	LA	80	Intern	1	152001734

```
In [38]: # Crear un filtre binari per als noms
binary_filter = pd.Series(
    {"Alice": True, "Bob": False, "Charlie": True, "David": True,
     "Eva": False, "Frank": False, "Grace": True})
binary_filter
```

```
Out[38]: Alice      True
Bob        False
Charlie    True
David      True
Eva        False
Frank      False
Grace      True
dtype: bool
```

```
In [39]: # Filtrar el DataFrame
df[binary_filter]
```

```
Out[39]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Charlie	40	SF	300	CEO	10	902939169
David	33	NY	250	Engineer	4	282966260
Grace	27	LA	80	Intern	1	152001734

Comparar columnes retorna un series booleà, així que també ho podem fer servir per filtrar!

```
In [40]: adults = df['Age'] > 35
adults
```

```
Out[40]: Name
Alice      False
Bob        False
Charlie    True
David      False
Eva        True
Frank      True
Grace      False
Name: Age, dtype: bool
```

```
In [41]: # Filtrar dades
df[adults]
```

```
Out[41]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Charlie	40	SF	300	CEO	10	902939169
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963

Múltiples filtres

- Utilitza `&` per a operacions AND.
- Utilitza `|` per a operacions OR.
- Els parèntesis són crucials per a múltiples condicions.

In [42]:

```
df
```

Out[42]:

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Bob	35	LA	200	Manager	5	110996078
Charlie	40	SF	300	CEO	10	902939169
David	33	NY	250	Engineer	4	282966260
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963
Grace	27	LA	80	Intern	1	152001734

```
In [43]: # Obtenir elements amb edat > 20 i ciutat = NY
df[(df['Age'] > 20) & (df['City'] == 'NY')]
```

```
Out[43]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
David	33	NY	250	Engineer	4	282966260
Eva	40	NY	350	Manager	12	803645337

```
In [44]: # Obtenir elements amb edat > 28 o salari <= 300
df[(df['Age'] > 28) | (df['Salary'] <= 300)]
```

```
Out[44]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Bob	35	LA	200	Manager	5	110996078
Charlie	40	SF	300	CEO	10	902939169
David	33	NY	250	Engineer	4	282966260
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963
Grace	27	LA	80	Intern	1	152001734

Ordenar

```
In [45]: # Ordena els valors ascendentment per defecte  
df.sort_values('Age')
```

```
Out[45]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Grace	27	LA	80	Intern	1	152001734
David	33	NY	250	Engineer	4	282966260
Bob	35	LA	200	Manager	5	110996078
Charlie	40	SF	300	CEO	10	902939169
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963

```
In [46]: # Ordena els valors descendentment
df.sort_values('Age', ascending=False)
```

Out[46]:

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Frank	55	SF	500	Engineer	15	077507963
Eva	40	NY	350	Manager	12	803645337
Charlie	40	SF	300	CEO	10	902939169
Bob	35	LA	200	Manager	5	110996078
David	33	NY	250	Engineer	4	282966260
Alice	27	NY	100	Engineer	2	787167724
Grace	27	LA	80	Intern	1	152001734

```
In [47]: # Ordena per múltiples columnes
df.sort_values(['Age', 'Salary'], ascending=False)
```

```
Out[47]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Frank	55	SF	500	Engineer	15	077507963
Eva	40	NY	350	Manager	12	803645337
Charlie	40	SF	300	CEO	10	902939169
Bob	35	LA	200	Manager	5	110996078
David	33	NY	250	Engineer	4	282966260
Alice	27	NY	100	Engineer	2	787167724
Grace	27	LA	80	Intern	1	152001734

```
In [48]: # Ordena amb direccions diferents per columna
df.sort_values(['Age', 'Salary'], ascending=[False, True])
```

```
Out[48]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Frank	55	SF	500	Engineer	15	077507963
Charlie	40	SF	300	CEO	10	902939169
Eva	40	NY	350	Manager	12	803645337
Bob	35	LA	200	Manager	5	110996078
David	33	NY	250	Engineer	4	282966260
Grace	27	LA	80	Intern	1	152001734
Alice	27	NY	100	Engineer	2	787167724

```
In [49]: # Ordenar per index
df.sort_index()
```

Out[49]:

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Bob	35	LA	200	Manager	5	110996078
Charlie	40	SF	300	CEO	10	902939169
David	33	NY	250	Engineer	4	282966260
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963
Grace	27	LA	80	Intern	1	152001734

Mètodes de strings

```
In [50]: # Tractar una columna de strings com a "strings normals"  
# accedint per index o fent slicing  
df['City'].str[0]
```

```
Out[50]:
```

Name	
Alice	N
Bob	L
Charlie	S
David	N
Eva	N
Frank	S
Grace	L

Name: City, dtype: object

```
In [51]: # Columna de strings 'City' a minúscules  
df['City'].str.lower()
```

```
Out[51]:
```

Name	
Alice	ny
Bob	la
Charlie	sf
David	ny
Eva	ny
Frank	sf
Grace	la

Name: City, dtype: object

Si no s'especifica el contrari, el DF no canvia:

```
In [52]: # Les ciutats encara estan en majúscules
df
```

```
Out[52]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Bob	35	LA	200	Manager	5	110996078
Charlie	40	SF	300	CEO	10	902939169
David	33	NY	250	Engineer	4	282966260
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963
Grace	27	LA	80	Intern	1	152001734

```
In [53]: # Canviar la ciutat d'en Bob
df.loc['Bob', 'City'] = 'NY'
```

```
In [54]: df
```

```
Out[54]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	NY	100	Engineer	2	787167724
Bob	35	NY	200	Manager	5	110996078
Charlie	40	SF	300	CEO	10	902939169
David	33	NY	250	Engineer	4	282966260
Eva	40	NY	350	Manager	12	803645337
Frank	55	SF	500	Engineer	15	077507963
Grace	27	LA	80	Intern	1	152001734

Moltes funcions del DF tenen un paràmetre `inplace`, utilitzat quan volem mantenir els canvis al DF.

```
In [55]: # Substituir valors en una columna
new_names = {'NY': 'New York', 'SF': 'San Francisco'}
df.replace({'City': new_names}, inplace=True)
```

```
In [56]: df
```

```
Out[56]:
```

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	New York	100	Engineer	2	787167724
Bob	35	New York	200	Manager	5	110996078
Charlie	40	San Francisco	300	CEO	10	902939169
David	33	New York	250	Engineer	4	282966260
Eva	40	New York	350	Manager	12	803645337
Frank	55	San Francisco	500	Engineer	15	077507963
Grace	27	LA	80	Intern	1	152001734

Podem encadenar tants mètodes com vulguem:

```
In [57]: # Posa-ho en majúscules i substitueix els espais per guions baixos  
df['City'].str.upper().str.replace(' ', '_')
```

```
Out[57]:
```

Name	
Alice	NEW_YORK
Bob	NEW_YORK
Charlie	SAN_FRANCISCO
David	NEW_YORK
Eva	NEW_YORK
Frank	SAN_FRANCISCO
Grace	LA

Name: City, dtype: object

Altres funcions

```
In [58]: # Suma dels salaris  
df['Salary'].sum()
```

```
Out[58]: np.int64(1780)
```

```
In [59]: # Edat mitjana  
df['Age'].mean()
```

```
Out[59]: np.float64(36.714285714285715)
```

```
In [60]: # Desviació estàndard de l'edat  
df['Age'].std()
```

```
Out[60]: np.float64(9.673232581274398)
```

```
In [61]: # Desviació estàndard de l'edat arrodonida  
df['Age'].std().round(2)
```

```
Out[61]: np.float64(9.67)
```

```
In [62]: # Valors únics  
df['City'].unique()
```

```
Out[62]: array(['New York', 'San Francisco', 'LA'], dtype=object)
```

```
In [63]: # Nombre de valors únics  
df['City'].nunique()
```

```
Out[63]: 3
```

```
In [64]: # Nombre d'elements per cada valor únic  
df['City'].value_counts()
```

```
Out[64]: City  
New York      4  
San Francisco  2  
LA            1  
Name: count, dtype: int64
```



```
In [65]: df['Age'] > 30
```

```
Out[65]:
```

Name	
Alice	False
Bob	True
Charlie	True
David	True
Eva	True
Frank	True
Grace	False

Name: Age, dtype: bool

```
In [66]: # Algun?  
(df['Age'] > 30).any()
```

```
Out[66]: np.True_
```

```
In [67]: # Tots?  
(df['Age'] > 30).all()
```

```
Out[67]: np.False_
```

```
In [68]: # Comprovar si està a ...  
df[df['City'].isin(['New York', 'San Francisco'])]
```

Out[68]:

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Alice	27	New York	100	Engineer	2	787167724
Bob	35	New York	200	Manager	5	110996078
Charlie	40	San Francisco	300	CEO	10	902939169
David	33	New York	250	Engineer	4	282966260
Eva	40	New York	350	Manager	12	803645337
Frank	55	San Francisco	500	Engineer	15	077507963

In [69]: *# Extreure dades aleatòriament del DF*
df.sample(4)

Out[69]:

	Age	City	Salary	Position	Years of Experience	Phone
Name						
Charlie	40	San Francisco	300	CEO	10	902939169
Grace	27	LA	80	Intern	1	152001734
Eva	40	New York	350	Manager	12	803645337
Bob	35	New York	200	Manager	5	110996078

```
In [70]: # Eliminar columna
df.drop(columns=['Phone'], inplace=True)
df
```

Out[70]:

	Age	City	Salary	Position	Years of Experience
Name					
Alice	27	New York	100	Engineer	2
Bob	35	New York	200	Manager	5
Charlie	40	San Francisco	300	CEO	10
David	33	New York	250	Engineer	4
Eva	40	New York	350	Manager	12
Frank	55	San Francisco	500	Engineer	15
Grace	27	LA	80	Intern	1

```
In [71]: # Eliminar fila per index  
# Sense "inplace" retorna un nou DF  
# però no modifica l'original!  
df.drop(index=['Charlie'])
```

```
Out[71]:
```

	Age	City	Salary	Position	Years of Experience
Name					
Alice	27	New York	100	Engineer	2
Bob	35	New York	200	Manager	5
David	33	New York	250	Engineer	4
Eva	40	New York	350	Manager	12
Frank	55	San Francisco	500	Engineer	15
Grace	27	LA	80	Intern	1

```
In [72]: df.rename(columns={'Years of Experience': 'Years Exp.'})
```

Out[72]:

	Age	City	Salary	Position	Years Exp.
Name					
Alice	27	New York	100	Engineer	2
Bob	35	New York	200	Manager	5
Charlie	40	San Francisco	300	CEO	10
David	33	New York	250	Engineer	4
Eva	40	New York	350	Manager	12
Frank	55	San Francisco	500	Engineer	15
Grace	27	LA	80	Intern	1

```
In [73]: # Selecciona columnes d'un tipus específic
df.select_dtypes('int')
```

```
Out[73]:
```

	Age	Salary	Years of Experience
Name			
Alice	27	100	2
Bob	35	200	5
Charlie	40	300	10
David	33	250	4
Eva	40	350	12
Frank	55	500	15
Grace	27	80	1

```
In [74]: # Selecciona columnes amb qualsevol tipus numèric
df.select_dtypes(include='number')
```

Out[74]:

	Age	Salary	Years of Experience
Name			
Alice	27	100	2
Bob	35	200	5
Charlie	40	300	10
David	33	250	4
Eva	40	350	12
Frank	55	500	15
Grace	27	80	1


```
In [75]: # Selecciona columnes amb "classes"
df.select_dtypes('object')
```

```
Out[75]:
```

	City	Position
Name		
Alice	New York	Engineer
Bob	New York	Manager
Charlie	San Francisco	CEO
David	New York	Engineer
Eva	New York	Manager
Frank	San Francisco	Engineer
Grace	LA	Intern

Llegir fitxers

Llegeix fitxers de diferents formats, p. ex. CSV o Excel.

```
In [ ]: # Llegir d'un fitxer CSV (no funciona perquè el fitxer no existeix)
df = pd.read_csv('data.csv', sep='|', parse_dates=['date_time'])
```

```
In [ ]: # Llegir d'un fitxer Excel (no funciona perquè el fitxer no existeix)
df = pd.read_excel('data.xlsx')
```